Cortés Bolaños Angel David - 423066940

Reporte Laboratorio S08

Parte 1: El Problema de las Transacciones Distribuidas y el Patrón Saga Orquestada

El Problema de la Inconsistencia en Microservicios

En una arquitectura monolítica, operaciones como una transferencia bancaria se ejecutan dentro de una única transacción de base de datos bajo los principios ACID, garantizando que el cargo y el abono ocurran juntos o se deshagan por completo. Sin embargo, en nuestro ecosistema de microservicios, los dominios de transferencia y cuenta están separados en bases de datos distintas, exponiendo al sistema a fallas parciales de red o disponibilidad.

Como se demuestra en el escenario inicial, la carencia de un manejador de transacciones distribuido produce pérdida de datos e inconsistencias críticas. Al iniciar la transferencia, el sistema logró procesar el cargo a la cuenta origen (reduciendo el saldo de Alicia de $1000 a $500), pero al apagar abruptamente el servicio de destino antes de concretar el abono a Roberto (saldo de $0), el flujo se interrumpió. El resultado fue una transacción incompleta donde el dinero desapareció del registro global, dejando el sistema en un estado corrompido.

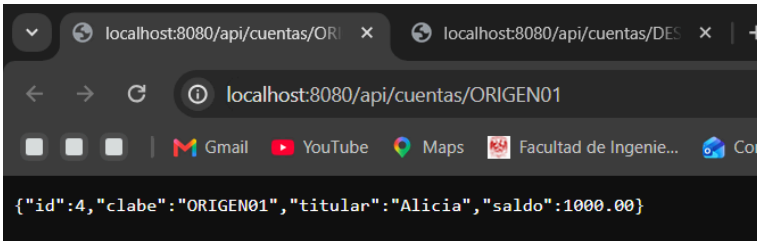


Figura 1: Alicia tiene 1000 en su cuenta

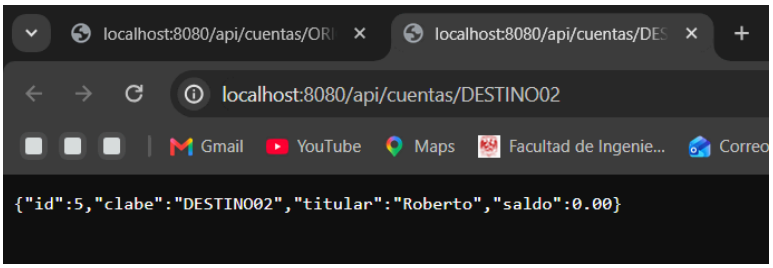


Figura 2: Roberto tiene 0 en su cuenta

Se realiza una transacción de 500 desde la cuenta de Alicia hacia Roberto

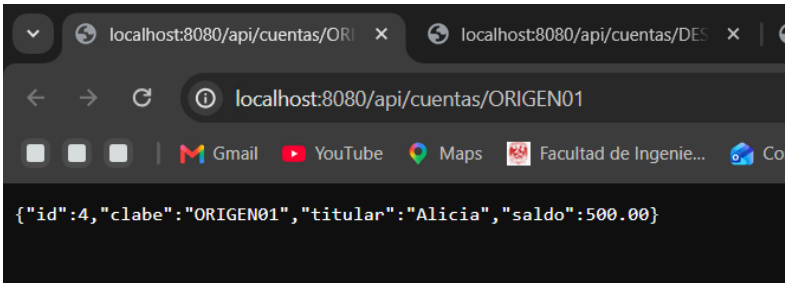


Figura 3: Se le restan los 500 a Alicia

En este momento se apaga el servicio de destino

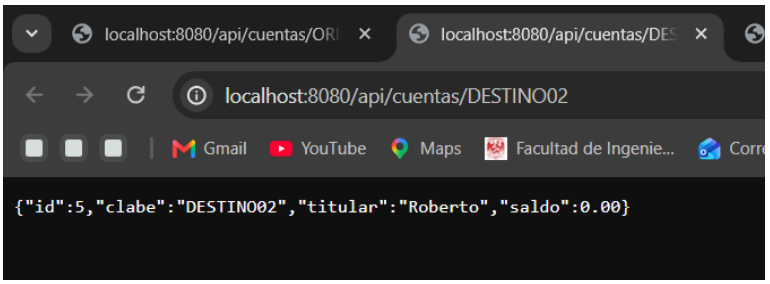


Figura 4: No cae el dinero en la cuenta de Roberto sigue en 0

Solución: Implementación del Patrón Saga Orquestada (Reverso Exitoso)
Para dotar al sistema de consistencia eventual, se implementó el Patrón Saga Orquestada. En este esquema, el microservicio de transferencias asume el rol de orquestador central, gestionando cada paso de la transacción distribuida a través de una máquina de estados explícita (INICIADA, CARGADA, COMPENSADA y COMPENSACION_PENDIENTE).

La principal característica de este patrón es la compensación atómica. Si un paso del proceso falla (como el abono), el orquestador intercepta el error y ejecuta en reversa las operaciones previamente completadas para restaurar el estado original. Para demostrar esta resiliencia, se forzó un fallo intentando transferir saldo a una cuenta inexistente (FALSO99). Como muestran las evidencias, el sistema dedujo el dinero de la cuenta origen pero, al detectar la excepción en el servicio de destino, abortó el flujo normal y disparó la lógica de reverso.

Al consultar la base de datos tras el fallo, se pudo constatar que el saldo de Alicia fue restituido íntegramente a los $1000 originales y la máquina de estados actualizó la transferencia como COMPENSADA.

Adicionalmente, se demostró la robustez del diseño frente a fallas catastróficas (como la caída total del servicio de cuentas que impide el propio reverso). En este escenario, el orquestador actuó como red de seguridad guardando la transacción con el estado de COMPENSACION_PENDIENTE. Esto garantiza que los datos no se pierdan y permite que, posteriormente, un proceso en segundo plano reintente devolver el dinero de forma asíncrona cuando el servicio se restablezca.

En TransferenciaService.java, se envolvió la llamada del abono en un bloque try/catch para atrapar el error e iniciar la compensación. Si el reverso falla, se captura esa segunda excepción para marcar el estado como pendiente.

```
try {
    // Intento de Abono
    destino = cuentas.abonar(claveDestino, monto);
} catch (Exception excepcionAbono) {
    try {
        // Compensación: Devolver el dinero al origen
        origen = cuentas.abonar(claveOrigen, monto);
        transferencia.setEstado(Estado.COMPENSADA);
    } catch (Exception excepcionReverso) {
        // Falló la compensación (Doble fallo)
        transferencia.setEstado(Estado.COMPENSACION_PENDIENTE);
    }
}
```

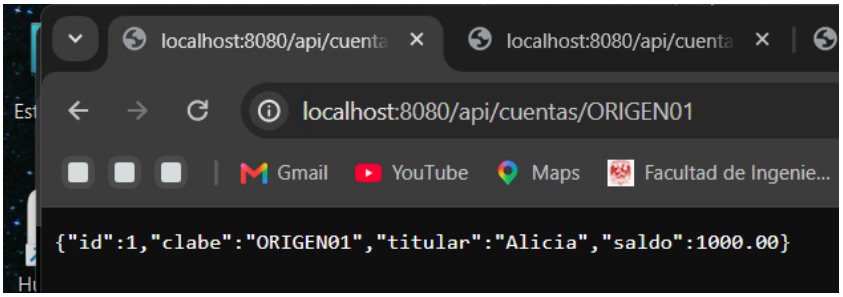


Figura 5: Nuevamente la cuenta de Alicia inicia en 1000

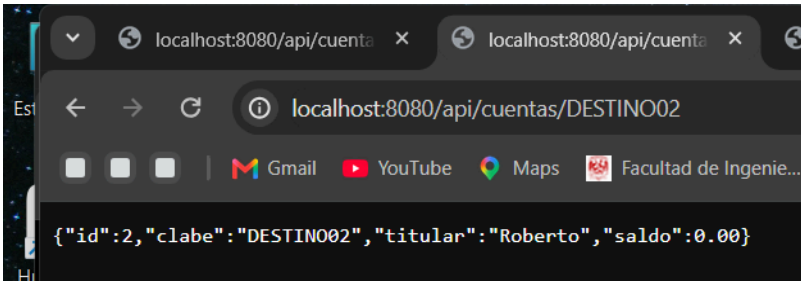


Figura 6: La cuenta de Roberto en 0

En este momento se apaga el servicio

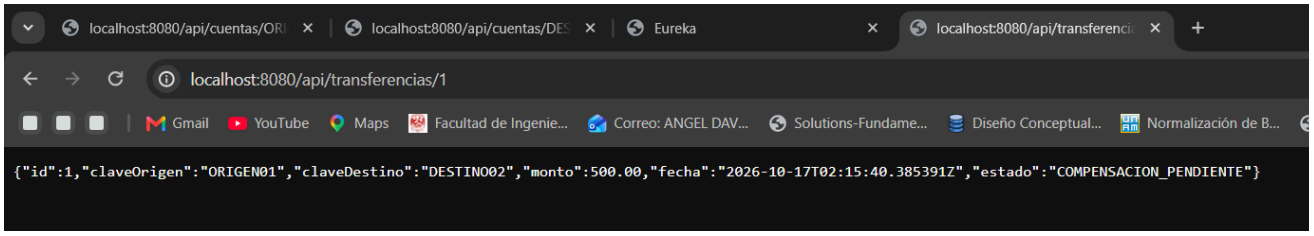


Figura 7: El dinero queda en una compensación pendiente

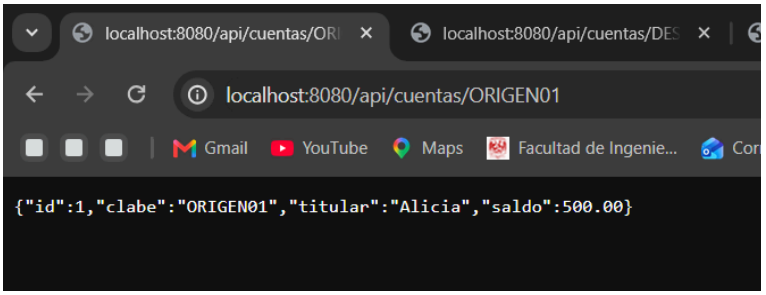


Figura 8: La cuenta de Alicia muestra 500 (todavía no se hace la compensación)

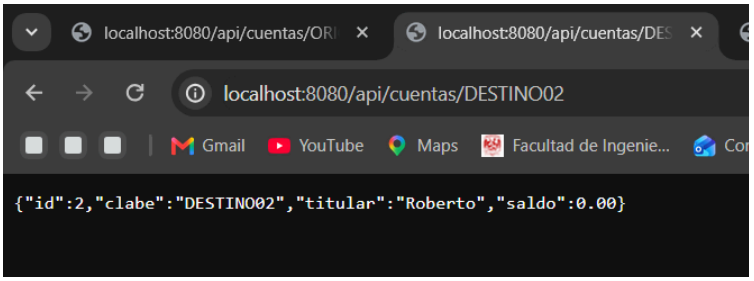


Figura 9: La cuenta sigue en 0 no le llegó la transferencia

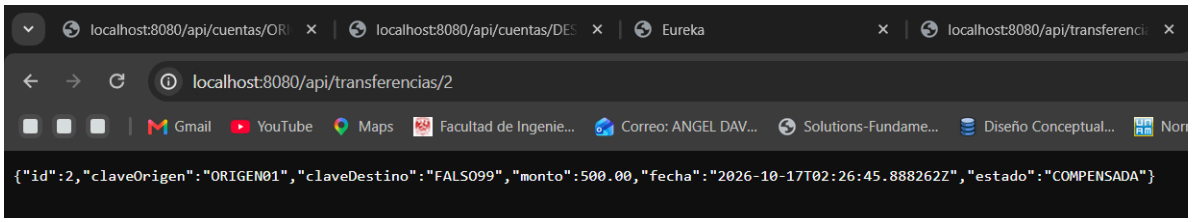
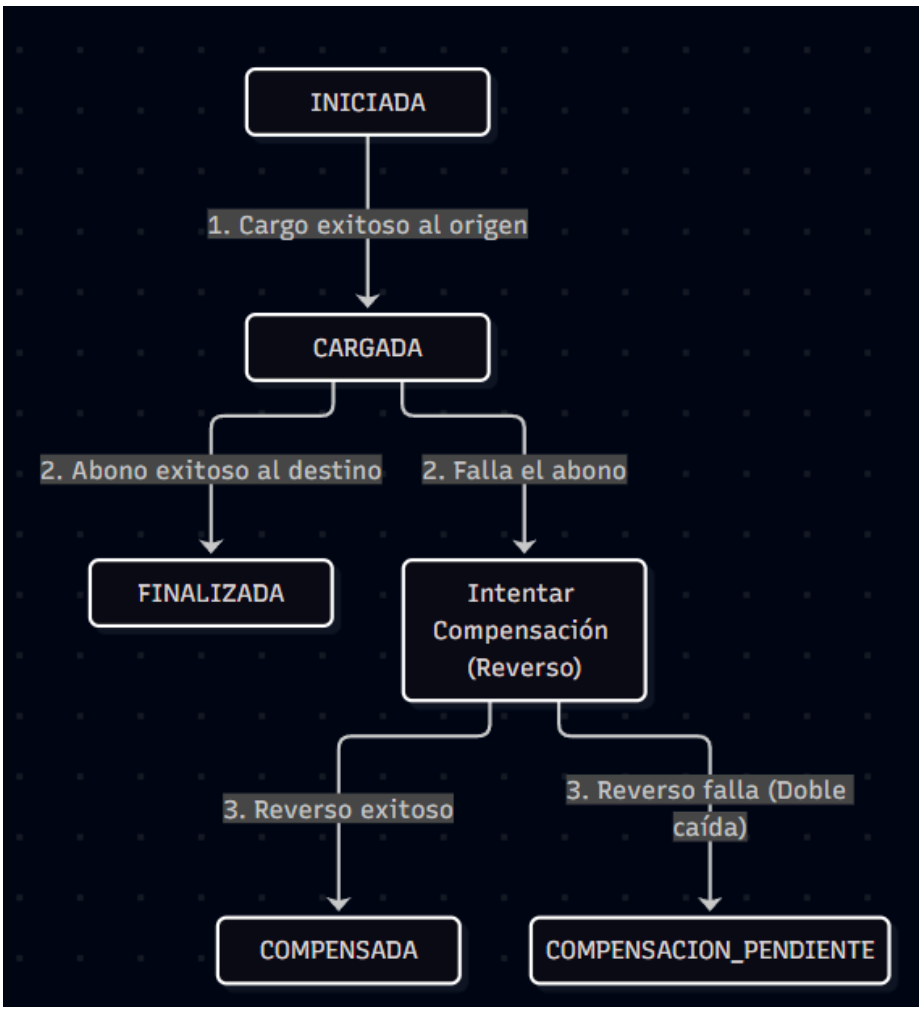


Figura 10: Aquí ya se hizo la compensación y la cuenta de Alicia regresó a 1000

Pasos de la Saga y su Compensación:

- **Paso 1 (Local):** Guardar transferencia en estado INICIADA. (Compensación: No aplica, si falla no hay efectos secundarios).
- **Paso 2 (Remoto):** Cargar al origen. El estado pasa a CARGADA. (Compensación: Hacer un abono remoto al origen por el mismo monto para deshacer el cargo).
- **Paso 3 (Remoto):** Abonar al destino. (Compensación: Hacer un cargo al destino, aunque en nuestra práctica si el abono falla simplemente hacemos el reverso del origen).



Parte 2: Prevención de Fallas en Cascada con el Patrón Circuit Breaker

El Cortacircuitos (Resilience4j) y la Estrategia Fail-Fast

El segundo gran desafío resuelto en la práctica es la prevención de fallas en cascada. En un sistema distribuido, cuando un microservicio crítico como el de cuenta colapsa, los servicios dependientes como transferencia se quedan esperando indefinidamente hasta que se agota el tiempo límite de red (Timeout). Esta espera inmoviliza los hilos de ejecución y satura la memoria del servidor. Si llegan múltiples peticiones al mismo tiempo, el colapso de un solo microservicio terminará derribando toda la arquitectura.

Para mitigar este riesgo, se implementa el patrón Circuit Breaker (Cortacircuitos) utilizando la biblioteca de Spring Boot Resilience4j, envolviendo las comunicaciones HTTP de red dentro de la clase cliente. Se configuró el cortacircuitos con una ventana de tolerancia de 3 errores consecutivos.

El patrón Circuit Breaker opera internamente a través de tres estados fundamentales:

1. Cerrado (Closed): El sistema funciona normalmente. Las peticiones fluyen hacia el servicio de cuentas. Aquí evidenciamos los errores por tiempo de espera ("Falla normal de red").

2. Abierto (Open): Tras superar el umbral de errores (3 fallos configurados), el fusible salta. No se intenta contactar al servicio de cuentas y se retorna una excepción inmediata ("CORTACIRCUITOS ABIERTO"). Las capturas anexadas demuestran este estado.
3. Medio-Abierto (Half-Open): Después de un tiempo de espera (configurado a 20 segundos), el fusible deja pasar unas cuantas peticiones de prueba. Si son exitosas, asume que el servicio revivió y vuelve a estado "Cerrado"; si fallan, regresa a "Abierto".

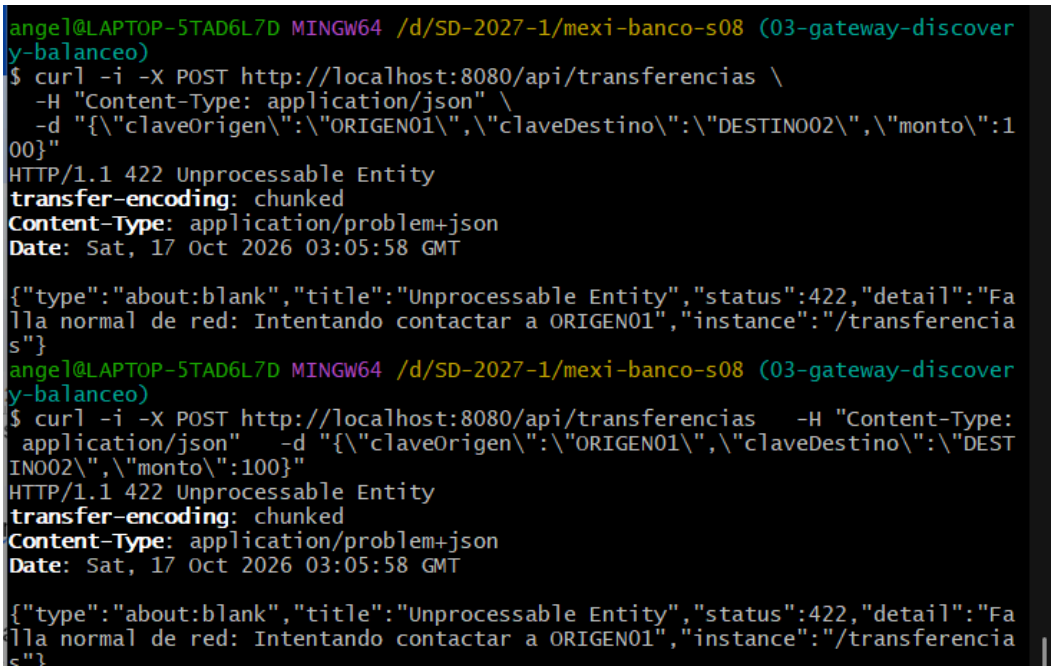


Figura 11. Primeras peticiones (se mantiene cerrado)

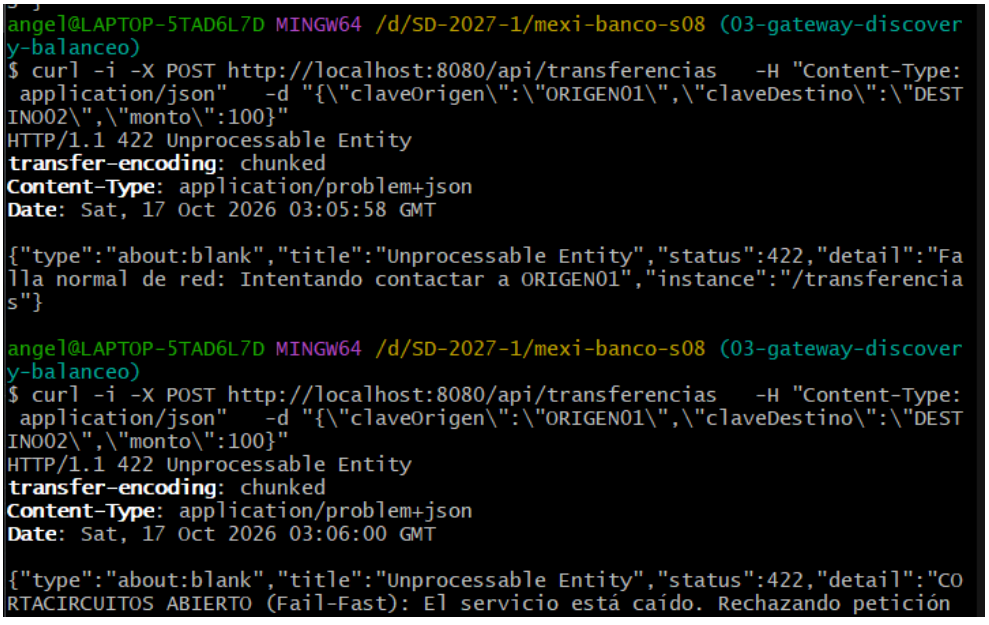


Figura 12. El circuito se abre

En CuentaCliente.java, se usó la anotación @CircuitBreaker para proteger las llamadas HTTP. Si el umbral de fallos se supera, Resilience4j invoca el método fallback para rechazar la petición inmediatamente.

```
@CircuitBreaker(name = "cuentaCB", fallbackMethod = "fallbackConsultar")
public CuentaRemota consultar(String clabe) {
    return http.get().uri("/cuentas/{clabe}", clabe).retrieve().body(CuentaRemota.class);
}

// Método de emergencia (Fallback)
public CuentaRemota fallbackConsultar(String clabe, Throwable t) {
    if (t.getClass().getSimpleName().equals("CallNotPermittedException")) {
        throw new ReglaDeNegocioException("CORTACIRCUITOS ABIERTO (Fail-Fast)...");
    }
    throw new ReglaDeNegocioException("Falla normal de red...");
}
```

Limitaciones de la Saga Orquestada: Falta de Aislamiento

A diferencia de las transacciones tradicionales (ACID), el Patrón Saga relaja la propiedad de Aislamiento a favor de la disponibilidad. Si un usuario o microservicio consulta el saldo de Alicia en el milisegundo exacto en que se ejecutó el cargo exitosamente, pero el abono en Roberto aún no falla (o está en proceso el reverso), ese usuario verá un saldo deducido. Los estados intermedios en una Saga son visibles para todo el ecosistema; por lo tanto, durante ese breve lapso, el sistema expone datos temporalmente incorrectos que podrían desencadenar lógicas o alertas basadas en información que eventualmente será compensada y revertida.

Conclusiones

La ejecución de esta práctica consolida la implementación de patrones arquitectónicos de alta disponibilidad. Por medio de la Saga Orquestada se logró garantizar integridad de datos sin depender de bloqueos estrictos de base de datos, automatizando la recuperación mediante estados compensatorios. De forma complementaria, el Circuit Breaker otorgó al sistema la madurez necesaria para aislar sus propias averías e implementar tolerancia a fallos, demostrando que en el ecosistema de los microservicios es vital saber "fallar de forma segura y rápida" para mantener vivo el resto del banco.